

The Challenge

This Write-Up is for the US Cyber Games Season V Beginner Game Room Crypto Challenge Block Blast.

Created by Elan Smolar Eisenberg, Sage Despeignes, Victoria Plinski

The challenge description is as follows:

Encryption can be powerful, but it often can be misused. Are you clever enough to unlock a secret and score the ultimate combo?

Solving the challenge

The participants are given a Python file called “challenge.py” when they run the code they get the following message:

```
=== AES-ECB Byte-at-a-Time Oracle ===  
Send hex-encoded bytes; I'll append my secret flag, pad, AES-encrypt, and  
give you the ciphertext (also hex). Empty line or 'exit' quits.  
>
```

Participants will then need to figure out how to interact with the oracle to get the flag. This should be done by sending a simple hex-encoded input such as:

```
4141414141414141414141414141414141
```

Which in turn results in the oracle responding with a ciphertext, which the participants should note down for further analysis.

How The Oracle Works

Based on the provided information in the code, the participants will know that the oracle uses AES in ECB mode, which is known to be vulnerable because identical plaintext blocks will produce identical ciphertext blocks. The participants are also tipped of via the code that they should perform a byte-at-a-time attack to get the flag.

The participants will also need to understand how the padding works. The oracle applies PKCS#7 padding, which means that the last byte of the plaintext indicates how many bytes are added for padding.

Crafting the Attack

To get the flag, participants will need to implement a byte-at-a-time attack. This is done by sending crafted inputs to the oracle to reveal the flag one byte at a time.

One way to do this is by creating a script to automate the process, the script would have three key steps:

1. Launch the Oracle
2. Send Inputs
3. Recover the Flag

To do the byte-at-a-time attack the script would need to do the following:

- Calculate the necessary padding length to align the input correctly.
- Construct a prefix determined by the padding calculation, to fill the block up to with point where the flag will be appended.
- Determine which block of ciphertext the participant is currently working with, based on the length of the prefix and the bytes of the flag that have been recovered.
- Retrieve and store the ciphertext of the current prefix, to then extract the target block of ciphertext that corresponds to the current block index.
- Iterate through all possible byte values to guess the next byte of the flag. For each guess, the script should construct a new input consisting of the prefix, the already recovered bytes, and the current guess. The input will then be sent to the oracle to retrieve the resulting ciphertext.
- If the ciphertext block generated from the guessed byte matches the target block, the guess is confirmed as correct, and the byte is appended to the `recovered` variable. The process continues until the entire flag is revealed.

After the script is ran the participants should get the flag:

SVBGR{M3G4_P0W3RUP_COMB0}